

Universidade Federal do Rio de Janeiro

Escola Politécnica



Universidade Federal
do Rio de Janeiro

Escola Politécnica

Projeto de um Sistema de Criptografia Baseado em One-Time Pad (OTP) Utilizando FPGA

Relatório Trabalho Prático 2

Disciplina de Sistemas Digitais

Alunos: Daniel Gudin
João Gabriel
Rayane Santos
Professor: Pedro Cruz
Universidade: Universidade Federal do Rio de Janeiro

Rio de Janeiro
2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Fundamentação Teórica	3
3.1	FPGA Spartan-3AN	3
3.2	Comunicação PS/2	3
3.3	Display LCD HD44780	3
3.4	Criptografia One-Time Pad	4
4	Arquitetura Geral do Sistema	5
5	Descrição dos Módulos	6
5.1	Módulo Principal (<code>crypto_top.vhd</code>)	6
5.2	Interface com o Teclado PS/2 (<code>kb_decoder.vhd</code>)	6
5.3	Buffers de Armazenamento	7
5.4	Módulo de Criptografia (<code>xor_engine.vhd</code>)	7
5.5	Conversor Byte para Hexadecimal (<code>hex_formatter.vhd</code>)	8
5.6	Controlador do Display LCD (<code>driver_lcd.vhd</code>)	8
6	Máquina de Estados	9
6.1	Estado de Inicialização	9
6.2	Estado de Entrada da Chave	9
6.3	Estado de Entrada da Mensagem	10
6.4	Estado de Criptografia	10
6.5	Estado de Conversão	10
6.6	Estado de Exibição	10
7	Fluxo de Funcionamento do Sistema	10
8	Interface com o Teclado PS/2	11
9	Interface com o Display LCD	11
10	Resultados Experimentais	12
11	Simulações e Validação do Núcleo Criptográfico	13
11.1	Simulação Avançada: Cadeia de Caracteres e Propriedades do OTP	14
12	Conclusão	15

1 Introdução

Os sistemas embarcados implementados em FPGA possibilitam a construção de soluções digitais de alto desempenho, permitindo que diferentes módulos de hardware operem simultaneamente de forma paralela. Essa característica torna as FPGAs extremamente adequadas para aplicações envolvendo processamento digital, controle de dispositivos externos, criptografia e protocolos de comunicação.

Neste trabalho foi desenvolvido um sistema completo de criptografia em linguagem VHDL para a FPGA Spartan-3AN, integrando comunicação com teclado PS/2, processamento dos dados em hardware e exibição das informações em um display LCD compatível com o controlador HD44780.

O projeto foi desenvolvido com o objetivo de atender aos requisitos propostos para a disciplina, nos quais o usuário deveria inserir informações através do teclado, o sistema deveria reconhecer corretamente cada tecla pressionada e os resultados deveriam ser apresentados no display da placa.

O funcionamento do sistema baseia-se na utilização do algoritmo de criptografia One-Time Pad (OTP), implementado através da operação lógica XOR entre os caracteres da mensagem e da chave fornecida pelo usuário. Após a realização da criptografia, cada byte resultante é convertido para sua representação hexadecimal e enviado ao display LCD.

Todo o controle do sistema é realizado por uma Máquina de Estados Finitos (Finite State Machine – FSM), responsável por coordenar a inicialização do display, a leitura das teclas, o armazenamento das informações, o processamento da criptografia e a atualização das informações apresentadas ao usuário.

O projeto foi dividido em módulos independentes, permitindo maior organização do código e facilitando a manutenção do sistema. Cada módulo possui uma função específica, como leitura do teclado, conversão de dados, criptografia ou controle do display, sendo todos integrados pelo módulo principal.

2 Objetivos

O principal objetivo deste projeto consiste no desenvolvimento de um sistema digital completo capaz de realizar criptografia em hardware utilizando uma FPGA.

Mais especificamente, buscou-se:

- Implementar uma interface de comunicação com um teclado PS/2;
- Identificar corretamente todas as teclas utilizadas durante a operação;
- Converter os Scan Codes enviados pelo teclado para caracteres ASCII;
- Armazenar separadamente uma chave criptográfica e uma mensagem digitada pelo usuário;
- Implementar em hardware o algoritmo One-Time Pad utilizando operações XOR;
- Converter os bytes criptografados para representação hexadecimal;
- Exibir as informações através do display LCD da placa;
- Integrar todos os módulos utilizando uma Máquina de Estados Finitos.

3 Fundamentação Teórica

3.1 FPGA Spartan-3AN

Uma FPGA (Field Programmable Gate Array) é um circuito integrado programável composto por blocos lógicos configuráveis e interconexões internas. Diferentemente de um microcontrolador, cujo funcionamento depende da execução sequencial de instruções de software, uma FPGA permite que diferentes circuitos operem simultaneamente.

Neste projeto foi utilizada a FPGA Spartan-3AN, programada em linguagem VHDL, permitindo a implementação totalmente em hardware dos módulos responsáveis pela comunicação com o teclado, processamento da criptografia e controle do display LCD.

A arquitetura modular utilizada torna possível que todos os blocos do sistema operem em paralelo, aumentando significativamente o desempenho da aplicação.

3.2 Comunicação PS/2

O teclado utilizado comunica-se através do protocolo PS/2.

Esse protocolo utiliza apenas duas linhas de comunicação:

- Clock (PS2C);
- Data (PS2D).

Sempre que uma tecla é pressionada, o teclado transmite um quadro serial contendo 11 bits:

- 1 bit de início;
- 8 bits de dados (Scan Code);
- 1 bit de paridade;
- 1 bit de parada.

O módulo `kb_decoder.vhd` implementa a recepção dos quadros seriais transmitidos pelo teclado PS/2, realizando a sincronização com o sinal de clock, a reconstrução do byte recebido e a identificação dos códigos de liberação de tecla (F0). Dessa forma, apenas os eventos correspondentes ao pressionamento das teclas são encaminhados aos demais módulos do sistema.

Após a recepção do *Scan Code*, o módulo realiza sua conversão para o código ASCII correspondente, permitindo que os demais módulos trabalhem diretamente com caracteres em vez de códigos do protocolo PS/2.

3.3 Display LCD HD44780

A saída do sistema é apresentada em um display LCD alfanumérico compatível com o controlador HD44780.

O display opera em modo de comunicação de quatro bits, reduzindo a quantidade de pinos utilizados na FPGA.

O módulo `driver_lcd.vhd` implementa toda a interface de comunicação com o display, sendo responsável pela sequência de inicialização do controlador HD44780, pelo controle dos tempos de comunicação e pelo envio de comandos e caracteres para o LCD.

Além disso, esse módulo controla os seguintes sinais de interface:

- RS (*Register Select*);
- RW (*Read/Write*);
- EN (*Enable*);
- DB4 a DB7 (barramento de dados em modo de 4 bits).

Após a inicialização, o controlador recebe os caracteres enviados pelo módulo principal e realiza sua escrita no display, permitindo que o usuário visualize as mensagens e o resultado da criptografia.

3.4 Criptografia One-Time Pad

O algoritmo utilizado baseia-se no conceito da cifra One-Time Pad (OTP).

A criptografia é realizada aplicando a operação lógica XOR entre cada caractere da mensagem e o caractere correspondente da chave.

Matematicamente,

$$C_i = M_i \oplus K_i$$

onde

- M_i representa o caractere da mensagem;
- K_i representa o caractere da chave;
- C_i representa o caractere criptografado.

Uma característica importante da operação XOR é que ela é reversível. Assim, aplicando novamente a mesma chave ao texto cifrado, obtém-se a mensagem original:

$$M_i = C_i \oplus K_i$$

No projeto desenvolvido, essa operação é implementada pelo módulo `otp_cipher.vhd`, responsável por realizar a criptografia byte a byte diretamente em hardware.

4 Arquitetura Geral do Sistema

O sistema foi desenvolvido seguindo uma arquitetura modular, na qual cada módulo desempenha uma função específica dentro do processo de criptografia. Essa organização facilita o desenvolvimento, a manutenção e a validação individual de cada bloco funcional, além de tornar o projeto mais organizado e reutilizável.

A Figura 1 apresenta o diagrama geral da arquitetura implementada.

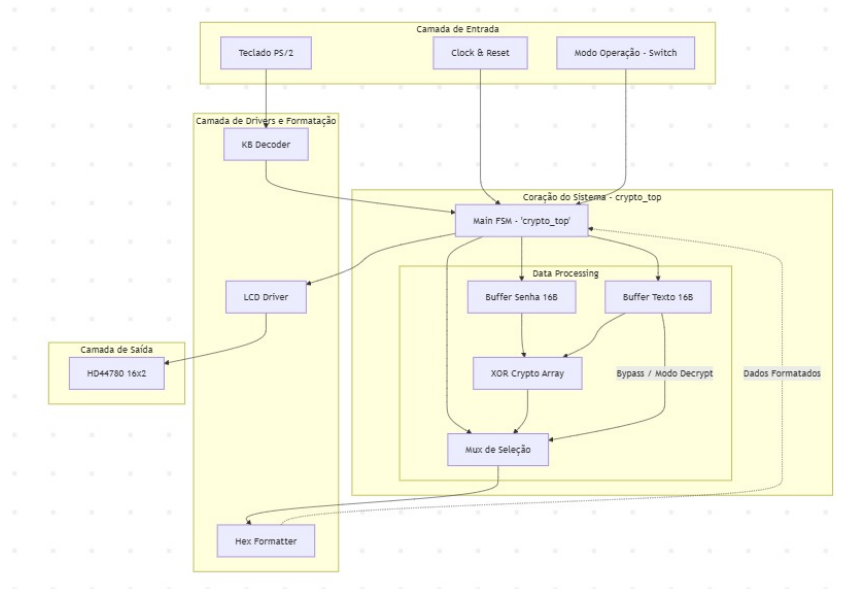


Figura 1: Arquitetura geral do sistema desenvolvido.

O funcionamento do sistema ocorre de acordo com a sequência descrita a seguir:

1. Inicialização do display LCD;
2. Recepção da chave criptográfica digitada pelo usuário;
3. Armazenamento da chave em um buffer interno;
4. Recepção da mensagem a ser criptografada;
5. Armazenamento da mensagem em um segundo buffer;
6. Processamento da criptografia utilizando a operação XOR;
7. Conversão dos bytes criptografados para representação hexadecimal;
8. Exibição do resultado no display LCD.

Todas essas etapas são coordenadas por uma Máquina de Estados Finitos (FSM), implementada no módulo principal do sistema.

5 Descrição dos Módulos

O sistema foi desenvolvido seguindo uma arquitetura modular composta por um módulo principal responsável pela integração dos demais módulos especializados. Cada bloco executa uma função específica, permitindo que o projeto seja organizado, de fácil manutenção e com maior facilidade para testes individuais.

5.1 Módulo Principal (`crypto_top.vhd`)

O módulo principal representa o núcleo do sistema, sendo responsável pela integração de todos os demais módulos implementados.

Entre os módulos conectados ao bloco principal destacam-se:

- Interface com o teclado PS/2;
- Controlador do display LCD;
- Conversor de bytes para hexadecimal;
- Módulo de criptografia baseado em XOR.

Além da interligação entre os módulos, este bloco implementa a Máquina de Estados Finitos (FSM), responsável por controlar toda a sequência de funcionamento do sistema.

As principais funções desempenhadas pelo módulo principal são:

- Inicializar o display LCD;
- Receber os caracteres provenientes do teclado PS/2;
- Diferenciar a entrada da chave criptográfica da entrada da mensagem;
- Armazenar os caracteres nos buffers internos;
- Detectar o pressionamento da tecla *ENTER*;
- Acionar o módulo de criptografia;
- Enviar os dados processados para o controlador do display LCD.

Durante toda a execução do sistema, o módulo principal monitora continuamente o sinal de tecla válida proveniente da interface PS/2. Sempre que um novo caractere é recebido, ele é armazenado na posição correspondente do buffer da chave ou da mensagem, de acordo com o estado atual da Máquina de Estados.

5.2 Interface com o Teclado PS/2 (`kb_decoder.vhd`)

O módulo `kb_decoder.vhd` implementa toda a comunicação entre a FPGA e o teclado PS/2.

Inicialmente, o módulo sincroniza o sinal de clock proveniente do teclado com o clock interno da FPGA, garantindo uma recepção confiável dos dados seriais. Em seguida, a cada borda de descida do clock PS/2, um novo bit é deslocado para um registrador interno até que todo o quadro de transmissão seja recebido.

Cada quadro enviado pelo teclado possui o seguinte formato:

Start	8 Bits de Dados	Paridade	Stop
-------	-----------------	----------	------

Após a recepção dos onze bits, o módulo verifica a validade do quadro recebido.

Caso seja identificado o código hexadecimal **F0**, correspondente à liberação de uma tecla, nenhuma ação é executada. Apenas os códigos referentes ao pressionamento efetivo das teclas são encaminhados para o restante do sistema.

Posteriormente, é realizada a conversão entre os *Scan Codes* do protocolo PS/2 e seus respectivos caracteres ASCII, permitindo que os demais módulos trabalhem diretamente com caracteres convencionais.

Ao término desse processo, é gerado um pulso indicando que uma nova tecla válida foi recebida pelo sistema.

5.3 Buffers de Armazenamento

O sistema utiliza dois buffers independentes para armazenar temporariamente as informações fornecidas pelo usuário.

O primeiro buffer é destinado ao armazenamento da chave criptográfica, enquanto o segundo armazena a mensagem que será submetida ao processo de criptografia.

Cada caractere recebido através do teclado é gravado sequencialmente na memória correspondente.

O término da digitação é identificado pelo pressionamento da tecla *ENTER*, fazendo com que a Máquina de Estados altere automaticamente sua operação para a etapa seguinte.

Essa estratégia garante que toda a mensagem esteja completamente armazenada antes do início do processo de criptografia.

5.4 Módulo de Criptografia (*xor_engine.vhd*)

Após a recepção completa da chave e da mensagem, o módulo *xor_engine.vhd* é habilitado para realizar a criptografia.

Seu funcionamento baseia-se exclusivamente na operação lógica XOR entre os bytes armazenados nos buffers de chave e mensagem.

Para cada posição da mensagem é realizada a operação

$$C_i = M_i \oplus K_i$$

onde

- M_i representa o caractere da mensagem;
- K_i representa o caractere correspondente da chave;
- C_i representa o caractere criptografado.

Por ser implementado diretamente em hardware, esse módulo realiza a operação de forma rápida e eficiente, explorando os recursos disponíveis na FPGA.

5.5 Conversor Byte para Hexadecimal (`hex_formatter.vhd`)

Após a criptografia, os bytes gerados podem assumir valores que não correspondem a caracteres ASCII imprimíveis.

Para permitir sua visualização no display LCD, cada byte é convertido para sua representação hexadecimal pelo módulo `hex_formatter.vhd`.

Inicialmente, cada byte é dividido em dois nibbles de quatro bits. Em seguida, cada nibble é convertido para o caractere hexadecimal correspondente.

Como exemplo,

01000001_2

é convertido para

41_{16}

permitindo que qualquer resultado da criptografia seja corretamente exibido no display.

5.6 Controlador do Display LCD (`driver_lcd.vhd`)

O módulo `driver_lcd.vhd` é responsável pela comunicação entre a FPGA e o display LCD alfanumérico compatível com o controlador HD44780.

A comunicação é realizada utilizando o modo de operação de quatro bits, no qual cada byte é dividido em dois nibbles e transmitido sequencialmente ao display, reduzindo a quantidade de pinos utilizados pela FPGA.

Além do barramento de dados, o módulo controla os sinais de interface:

- RS (*Register Select*);
- RW (*Read/Write*);
- EN (*Enable*).

Durante a inicialização do sistema, o controlador executa automaticamente a sequência de configuração do display, respeitando os tempos mínimos exigidos pelo controlador HD44780 para garantir seu correto funcionamento.

Entre os principais comandos enviados destacam-se:

- Configuração do modo de comunicação em quatro bits;
- Configuração para duas linhas de exibição;
- Limpeza do display;
- Posicionamento inicial do cursor.

Após a inicialização, o módulo permanece aguardando novas informações provenientes do módulo principal. Sempre que um novo caractere deve ser exibido, o controlador gera automaticamente toda a sequência de sinais necessária para sua escrita no LCD, abstraindo a complexidade da comunicação e permitindo que o módulo principal apenas forneça os dados a serem apresentados.

6 Máquina de Estados

Todo o funcionamento do sistema é coordenado por uma Máquina de Estados Finitos (Finite State Machine – FSM), implementada no módulo principal.

Essa máquina controla a sequência de execução do projeto, garantindo que cada etapa seja executada na ordem correta.

A Figura ?? apresenta o diagrama de estados utilizado no sistema.

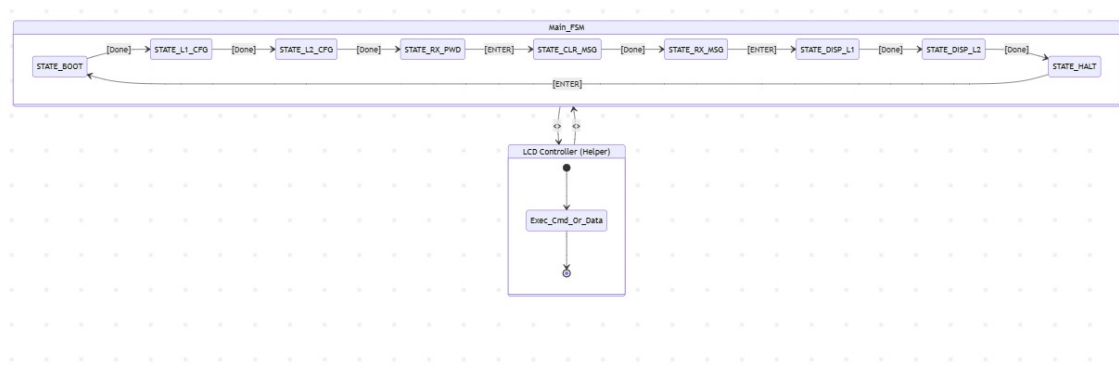


Figura 2: Diagrama de estados da Máquina de Estados Finitos (FSM) do sistema.

Os principais estados da FSM são descritos a seguir.

6.1 Estado de Inicialização

Após a configuração da FPGA, o sistema entra no estado de inicialização. Nesse estado ocorre:

- Reset interno dos registradores;
- Inicialização do display LCD;
- Limpeza dos buffers;
- Configuração das variáveis internas.

Somente após a conclusão dessa etapa o sistema passa a aceitar dados provenientes do teclado.

6.2 Estado de Entrada da Chave

Neste estado, todos os caracteres digitados pelo usuário são armazenados no buffer destinado à chave criptográfica.

Cada novo caractere recebido é inserido na próxima posição disponível do buffer.

A transição para o próximo estado ocorre quando o usuário pressiona a tecla ENTER.

6.3 Estado de Entrada da Mensagem

Após o armazenamento da chave, o sistema passa automaticamente para o estado responsável pela recepção da mensagem.

O funcionamento é semelhante ao da etapa anterior.

Cada caractere recebido é armazenado sequencialmente até que o usuário pressione novamente a tecla ENTER.

6.4 Estado de Criptografia

Com a chave e a mensagem completamente armazenadas, o módulo principal habilita o bloco `otp_cipher`.

Durante essa etapa é realizada uma operação XOR entre cada caractere da mensagem e o caractere correspondente da chave.

Os resultados produzidos são armazenados em um buffer interno.

6.5 Estado de Conversão

Os bytes gerados pela criptografia são enviados ao módulo `hex_converter.vhd`.

Cada byte é convertido em dois caracteres ASCII correspondentes aos dígitos hexadecimais.

Essa conversão torna possível a visualização de qualquer valor binário no display LCD.

6.6 Estado de Exibição

Após a conversão hexadecimal, o módulo principal envia os caracteres ao controlador do display.

O controlador escreve cada caractere na posição correspondente do LCD, permitindo ao usuário visualizar o resultado da criptografia.

Ao término da escrita, o sistema permanece aguardando uma nova operação ou retorna ao estado inicial, dependendo da lógica implementada.

7 Fluxo de Funcionamento do Sistema

O funcionamento do sistema pode ser descrito da seguinte maneira:

1. O sistema inicializa a FPGA e configura o display LCD;
2. O controlador do teclado permanece aguardando uma tecla válida;
3. O usuário digita a chave criptográfica;
4. Os caracteres são convertidos para ASCII e armazenados;
5. Após ENTER, inicia-se a leitura da mensagem;
6. Os caracteres da mensagem são armazenados em um segundo buffer;
7. O módulo OTP realiza a operação XOR entre mensagem e chave;
8. O resultado é convertido para hexadecimal;

9. O controlador do display apresenta os caracteres no LCD.

Observa-se que todo esse processo ocorre de forma automática, sendo controlado exclusivamente pela Máquina de Estados implementada no módulo principal.

8 Interface com o Teclado PS/2

A entrada de dados do sistema é realizada por meio de um teclado padrão PS/2 conectado diretamente à FPGA. Essa interface foi implementada no módulo `ps2_keyboard.vhd`, responsável por interpretar o protocolo de comunicação e disponibilizar os caracteres digitados para o restante do sistema.

O protocolo PS/2 utiliza comunicação serial síncrona, composta por duas linhas:

- **PS2C**: linha de clock gerada pelo teclado;
- **PS2D**: linha de dados.

O módulo receptor monitora continuamente o sinal de clock do teclado. A cada borda de descida, um novo bit é deslocado para um registrador interno até que todo o quadro seja recebido.

Após a recepção completa, o módulo verifica a validade do quadro e identifica o Scan Code correspondente à tecla pressionada.

Quando o teclado envia o código F0, indicando a liberação de uma tecla, esse código é descartado para evitar leituras duplicadas. Dessa forma, somente o evento de pressionamento é utilizado pelo sistema.

Posteriormente, os Scan Codes são convertidos para caracteres ASCII. O caractere resultante é enviado ao módulo principal juntamente com um sinal indicando que uma nova tecla válida foi recebida.

9 Interface com o Display LCD

A saída do sistema é apresentada em um display LCD alfanumérico compatível com o controlador HD44780.

O controlador implementado no módulo `display.vhd` opera em modo de quatro bits, reduzindo o número de pinos utilizados na FPGA.

A comunicação ocorre através dos sinais:

- RS (Register Select);
- RW (Read/Write);
- EN (Enable);
- DB4 a DB7 (Barramento de Dados).

Durante a inicialização, o controlador envia automaticamente a sequência de comandos necessária para configurar o display.

Após essa etapa, os caracteres recebidos do módulo principal são enviados ao LCD, permitindo ao usuário acompanhar todas as etapas da operação.

Dependendo do estado da máquina de estados, o display pode apresentar:

- Mensagens de inicialização;
- Chave digitada;
- Mensagem digitada;
- Resultado da criptografia em formato hexadecimal.

10 Resultados Experimentais

Após a implementação dos módulos em VHDL, o sistema foi sintetizado e programado na FPGA Spartan-3AN.

Os testes realizados tiveram como objetivo verificar o correto funcionamento de todas as etapas do projeto.

Foram avaliadas as seguintes funcionalidades:

- Inicialização correta do display LCD;
- Comunicação entre FPGA e teclado PS/2;
- Conversão correta dos Scan Codes para ASCII;
- Armazenamento da chave criptográfica;
- Armazenamento da mensagem;
- Execução da operação XOR;
- Conversão hexadecimal dos bytes criptografados;
- Atualização correta do display LCD.

Durante os testes experimentais, observou-se que todas as funcionalidades previstas foram executadas corretamente.

A recepção dos caracteres ocorreu sem perda de informações, os buffers armazenaram corretamente os dados digitados e o módulo de criptografia produziu resultados compatíveis com aqueles obtidos através de cálculos realizados em software.

Além disso, verificou-se que a conversão para hexadecimal realizada pelo módulo `hex_formatter.vhd` permitiu representar corretamente qualquer byte gerado pelo processo de criptografia, possibilitando sua exibição no display LCD, mesmo quando o valor produzido não correspondia a um caractere ASCII imprimível.

Após o término da digitação da mensagem, o módulo OTP realiza a operação XOR entre os caracteres correspondentes.

Os bytes produzidos são convertidos para hexadecimal e enviados ao display LCD.

Esse procedimento foi repetido com diferentes combinações de chaves e mensagens, confirmando o correto funcionamento do algoritmo implementado.

11 Simulações e Validação do Núcleo Criptográfico

Para validar o funcionamento lógico do módulo combinacional de criptografia (`otp_cipher`), foi realizada uma simulação temporal de forma de onda (*waveform*), focada especificamente na operação da cifra *One-Time Pad* (XOR).

A simulação ilustrada na Figura 3 demonstra a propriedade fundamental da cifra XOR: a simetria entre os processos de cifragem e decifragem utilizando o mesmo hardware. O teste foi dividido em dois períodos exatos de 10 nanossegundos (ns), descritos a seguir:



Figura 3: Forma de onda demonstrando a cifragem e decifragem de um caractere via operação XOR.

Fase 1 (0 a 10 ns): Cifragem da Mensagem

No primeiro instante da simulação, o objetivo é mascarar a informação original.

- **Sinal de Entrada (`entrada_dados`):** É injetado no sistema o valor hexadecimal `0x41`, que corresponde à letra 'A' na tabela ASCII.
- **Sinal da Chave (`chave_secreta`):** Aplica-se a chave criptográfica com o valor hexadecimal `0x42` (letra 'B').
- **Sinal de Saída (`saida_dados`):** Imediatamente (considerando o atraso combinacional ideal nulo na simulação), o barramento de saída avalia o resultado da operação XOR lógico ($0x41 \oplus 0x42$), resultando no valor `0x03`. Este valor representa o caractere de controle *End of Text* (ETX) na tabela ASCII, que é não-imprimível/invisível em um teclado comum. Isso comprova que a mensagem original foi criptografada com sucesso, tornando-se ininteligível.

Fase 2 (10 a 20 ns): Decifragem e Recuperação da Mensagem

No segundo instante, demonstra-se a recuperação da informação original submetendo o texto cifrado ao mesmo circuito.

- **Sinal de Entrada (`entrada_dados`):** O sinal é atualizado para `0x03`, realimentando o sistema com o texto cifrado gerado na etapa anterior.
- **Sinal da Chave (`chave_secreta`):** A chave é mantida constante no valor `0x42` (letra 'B').
- **Sinal de Saída (`saida_dados`):** No exato momento em que ocorre a transição dos sinais de entrada, a saída computa a nova operação lógica ($0x03 \oplus 0x42$) e retorna imediatamente para o valor `0x41`.

Essa transição prova a premissa matemática da cifra de Vernam (OTP), onde $(M \oplus K) \oplus K = M$. A letra 'A' foi recuperada intacta, atestando que a mesma arquitetura de hardware utilizada para criptografar é capaz de descriptografar a mensagem, eliminando a necessidade de desenvolver circuitos paralelos ou de alocar hardware adicional para o processo reverso.

11.1 Simulação Avançada: Cadeia de Caracteres e Propriedades do OTP

Para corroborar a estabilidade do circuito processando uma sequência de dados e evidenciar uma das principais forças do algoritmo *One-Time Pad*, elaborou-se uma segunda análise temporal, detalhada na Figura 4. Neste cenário, avalia-se a cifragem e decifragem da palavra "OLA" utilizando a chave sequencial "123".

Primeiramente, estabelece-se o mapeamento dos caracteres para seus respectivos valores na tabela ASCII, em base hexadecimal:

- **Texto Claro (Mensagem):** 'O' = 0x4F, 'L' = 0x4C, 'A' = 0x41.
- **Chave Criptográfica:** '1' = 0x31, '2' = 0x32, '3' = 0x33.

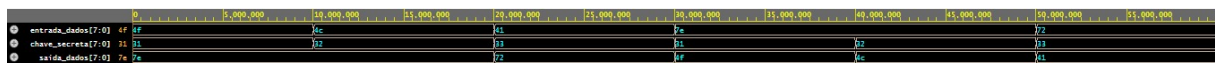


Figura 4: Forma de onda processando a cifragem e decifragem de múltiplos caracteres, evidenciando a robustez contra análise de frequência.

Fase 1 (0 a 30 ns): Cifragem e Quebra de Padrões Estatísticos

Durante a primeira metade da simulação, o barramento de entrada (`entrada_dados`) recebe a mensagem original, enquanto o barramento de chave (`chave_secreta`) é alimentado sincronamente com os caracteres da senha.

- **Aos 0 ns:** A operação XOR entre 'O' (0x4F) e a chave '1' (0x31) resulta no valor 0x7E.
- **Aos 10 ns:** A operação entre 'L' (0x4C) e a chave '2' (0x32) também resulta no valor 0x7E.
- **Aos 20 ns:** A operação entre 'A' (0x41) e a chave '3' (0x33) resulta no valor 0x72.

Análise Crítica: É imperativo notar o comportamento do sistema nos instantes 0 ns e 10 ns. Letras originais distintas ('O' e 'L') geraram exatamente o mesmo código criptografado (0x7E). Esse fenômeno ilustra a principal robustez matemática do OTP: a destruição da morfologia da mensagem original. Ao combinar os dados com uma chave variante, o sistema impede ataques baseados em análise de frequência, uma vez que a repetição de caracteres no texto cifrado não implica repetição no texto claro, e vice-versa. O texto foi transformado em ruído uniforme.

Fase 2 (30 a 60 ns): Decifragem e Verificação de Estabilidade

Na segunda metade da análise temporal, demonstra-se a reversibilidade perfeita do sistema, injetando a sequência de texto cifrado ininteligível (0x7E, 0x7E, 0x72) na entrada do circuito, mantendo a mesma chave.

- **Aos 30 ns:** O código cifrado 0x7E operado com a chave '1' (0x31) restaura corretamente o caractere original 'O' (0x4F).
- **Aos 40 ns:** O mesmo código cifrado 0x7E, agora operado com a chave '2' (0x32), decodifica para o caractere 'L' (0x4C).

- **Aos 50 ns:** O código 0x72 processado com a chave '3' (0x33) revela o caractere 'A' (0x41).

A mensagem original "OLA" foi integralmente recuperada. Este resultado valida a estabilidade da máquina lógica ao processar fluxos contínuos de dados e reafirma que a decodificação correta depende intrinsecamente do alinhamento posicional exato entre o texto cifrado e a chave correspondente.

12 Conclusão

Neste trabalho foi desenvolvido um sistema de criptografia em hardware utilizando a FPGA Spartan-3AN, integrando um teclado PS/2 para entrada de dados, um display LCD compatível com o controlador HD44780 para exibição das informações e um algoritmo de criptografia baseado na operação XOR, inspirado no conceito de One-Time Pad (OTP).

Os objetivos propostos foram alcançados com sucesso, uma vez que o sistema foi capaz de identificar corretamente as teclas pressionadas pelo usuário, armazenar a chave criptográfica e a mensagem em buffers internos, realizar a operação de criptografia em hardware e exibir o resultado no display LCD em formato hexadecimal. Dessa forma, o projeto atendeu integralmente aos requisitos estabelecidos para a disciplina.

A implementação modular em VHDL mostrou-se uma estratégia eficiente, permitindo a separação das funcionalidades em blocos independentes, como a interface com o teclado PS/2, o controlador do display LCD, o módulo de criptografia, o conversor para hexadecimal e o módulo principal responsável pelo controle da máquina de estados. Essa organização facilitou tanto o desenvolvimento quanto a validação individual de cada componente.

Além da implementação prática do sistema, o projeto possibilitou a aplicação de diversos conceitos estudados ao longo da disciplina, incluindo projeto de sistemas digitais em VHDL, desenvolvimento de máquinas de estados finitos, comunicação com dispositivos externos, manipulação de dados em hardware e integração de módulos em FPGA. As simulações e os testes realizados confirmaram o funcionamento correto de todos os blocos desenvolvidos, demonstrando a confiabilidade da solução implementada.

Por fim, o desenvolvimento deste projeto contribuiu significativamente para a consolidação dos conhecimentos sobre projeto digital em FPGA, proporcionando experiência prática no desenvolvimento de sistemas embarcados e evidenciando as vantagens da implementação de algoritmos diretamente em hardware, como maior paralelismo, rapidez de processamento e integração entre diferentes periféricos.

Referências

- [1] M. Morris Mano. *Digital Design*. Pearson.
- [2] Douglas Perry. *VHDL*. McGraw-Hill.
- [3] IBM. *Personal System/2 Hardware Interface Technical Reference*.
- [4] Hitachi. *HD44780U LCD Controller/Driver Datasheet*.
- [5] Xilinx Inc. *Spartan-3AN FPGA Family User Guide*.

Anexo A - Repositório do Projeto no GitHub

Os arquivos desenvolvidos para este trabalho, incluindo os códigos-fonte em VHDL, arquivos de configuração do projeto e demais recursos necessários para a implementação do sistema de criptografia baseado em One-Time Pad (OTP) utilizando FPGA, estão disponíveis em um repositório público no *GitHub*.

O repositório pode ser acessado por meio do endereço abaixo:

<https://github.com/Rayane-Sousa/fpga-otp-cryptography.git>